

TileKernels and TileLang Kernel Surfaces

Deep Dive into DeepSeek — Chapter 24

Yin Yang

Foundation Books Project

The one rule of this chapter

A numerically plausible tensor is **wrong** if any identity behind it has drifted.

- TileLang kernels may reorder rows, insert padding, and pack values — but the next operator still needs the right token, the right expert, and the right scale.
- **TileKernels** implements the operations that sit outside DeepGEMM matrix products, FlashMLA attention, and DeepEP communication: routing, per-token quantization, fused routed transforms, and token-order restoration.

Goal: follow a three-token route through quantization, gating, expert-major expansion, and reduction — then extend the same ownership discipline to Engram, mHC, and transpose — and see exactly what the released tests do and do not establish.

1. The running route, and what correctness means here

1. The running route, and what correctness means here
2. Ownership: program ids to the evidence ladder

1. The running route, and what correctness means here
2. Ownership: program ids to the evidence ladder
3. Quantization, gating, and mapping

1. The running route, and what correctness means here
2. Ownership: program ids to the evidence ladder
3. Quantization, gating, and mapping
4. Reduction and adjacent surfaces

1. The running route, and what correctness means here
2. Ownership: program ids to the evidence ladder
3. Quantization, gating, and mapping
4. Reduction and adjacent surfaces
5. Testing, evidence, and the chapter recap

**The running route, and what
correctness means here**

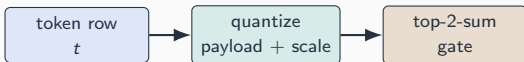
A three-token route, shrunk from a V3-style anchor

token row
 t

A three-token route, shrunk from a V3-style anchor



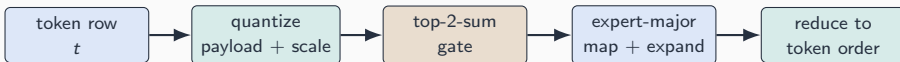
A three-token route, shrunk from a V3-style anchor



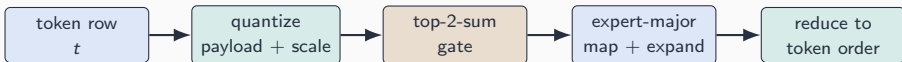
A three-token route, shrunk from a V3-style anchor



A three-token route, shrunk from a V3-style anchor

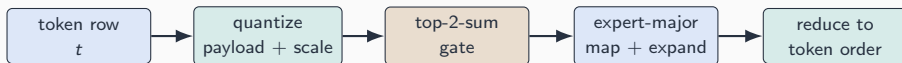


A three-token route, shrunk from a V3-style anchor



The public V3 architecture supplies the realistic anchor: hidden width 7168, 256 routed experts, top-8 activation, FP8 artifacts. The chapter's worked fixture shrinks this to three tokens and top-2 arithmetic, with $\text{topk_idx} = \begin{bmatrix} 2 & 0 \\ 1 & -1 \\ 0 & 2 \end{bmatrix}$ (-1 marks an invalid route slot) — small enough to check by hand at every stage.

A three-token route, shrunk from a V3-style anchor



The public V3 architecture supplies the realistic anchor: hidden width 7168, 256 routed experts, top-8 activation, FP8 artifacts. The chapter's worked fixture shrinks this to three tokens and top-2 arithmetic, with $\text{topk_idx} = \begin{bmatrix} 2 & 0 \\ 1 & -1 \\ 0 & 2 \end{bmatrix}$ (-1 marks an invalid route slot) — small enough to check by hand at every stage.

This fixture was never observed in a V3 execution — it is a derived teaching route, not a claim about production traffic.

What TileLang supplies, and what TileKernels adds

Definition (TileLang kernel surface)

A TileLang kernel surface is the semantic contract of a TileKernels kernel: the tensor relation it implements, its accepted shapes and dtypes, the launch ownership and synchronization it requires, and the tensors or metadata it returns.

- TileLang supplies the tile-programming language and compiler; TileKernels supplies the operator-specific kernels, their PyTorch wrappers, and their reference tests.
- A public implementation is not the same as confirmed production adoption. Released code and tests establish behavior on checked cases — not that DeepSeek's internal systems call them.

**Ownership: program ids to the
evidence ladder**

Program ownership, fragments, and shared state

- A **program id** names one launch instance's slice of ownership — one token block, one hidden block, one expert segment.
- A **fragment** is the on-chip tile a program id stages before a compute step; a **shared buffer** coordinates values across threads inside one program id.
- The per-token cast, the top-2-sum gate, and the fused expansion kernel are three different launch geometries built from the same four-level vocabulary: program id, fragment, shared buffer, mask.

The program id is a logical ownership claim, not a physical promise about which streaming multiprocessor executes it.

The evidence ladder: ownership before tolerance

Ladder step	What it establishes
Ownership identity	which program id, token, expert, and scale a value belongs to

The evidence ladder: ownership before tolerance

Ladder step	What it establishes
Ownership identity	which program id, token, expert, and scale a value belongs to
Shape and dtype	the tensor is even comparable to a reference

The evidence ladder: ownership before tolerance

Ladder step	What it establishes
Ownership identity	which program id, token, expert, and scale a value belongs to
Shape and dtype	the tensor is even comparable to a reference
Numerical tolerance	the payload is close enough under a stated metric

The evidence ladder: ownership before tolerance

Ladder step	What it establishes
Ownership identity	which program id, token, expert, and scale a value belongs to
Shape and dtype	the tensor is even comparable to a reference
Numerical tolerance	the payload is close enough under a stated metric

A tolerance comparison run before ownership identity is checked can pass on a tensor that is wrong — close values sitting in the wrong token's row. Every section from here on checks ownership first.

Quantization, gating, and mapping

E5M6: an exact packing format, not a rounding shortcut

- E5M6 packs **12 bits per value**: eight values pack into three uint32 words, with one scale owning an entire declared block.
- The unrounded scale is FP32; `round_sf` rounds it upward to a power of two, stored as FP32 or packed UE8M0.
- Fused SwiGLU backward returns BF16 and quantized input gradients, plus route-weight gradients, all under the *same* clamp and inverse-map state as the forward pass.

Payload and scale are one representation, not two — a payload compared without its scale, or a scale swapped between two otherwise correct rows, produces a wrong reconstruction that no shape check catches.

Gating, mapping, and a fused expert-major layout

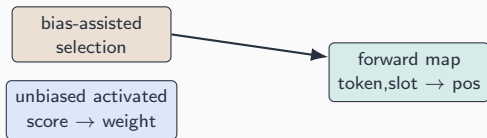
bias-assisted
selection

Gating, mapping, and a fused expert-major layout

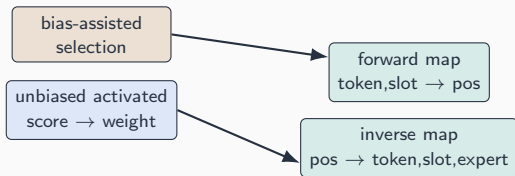
bias-assisted
selection

unbiased activated
score $\rightarrow$ weight

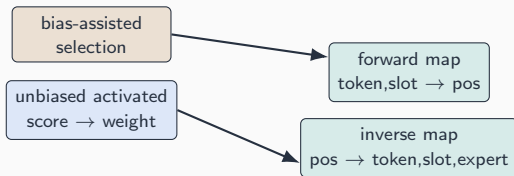
Gating, mapping, and a fused expert-major layout



Gating, mapping, and a fused expert-major layout

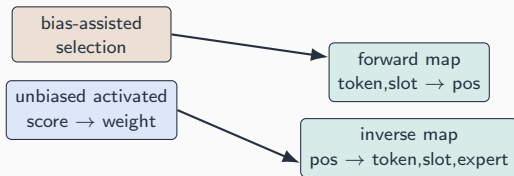


Gating, mapping, and a fused expert-major layout



- The top-2-sum gate's bias-assisted field only selects groups and experts; route weights come from the *unbiased* activated score, normalized separately.
- Logical expert ids, physical-duplicate ids, and rank-local ids are three distinct id spaces that a fused layout must keep distinguishable.

Gating, mapping, and a fused expert-major layout



- The top-2-sum gate's bias-assisted field only selects groups and experts; route weights come from the *unbiased* activated score, normalized separately.
- Logical expert ids, physical-duplicate ids, and rank-local ids are three distinct id spaces that a fused layout must keep distinguishable.

A fused expert-major layout is correct only when every valid route slot's forward map and inverse map agree — on the same token, the same slot, and the same expert.

Reduction and adjacent surfaces

Closing the loop: reduction as inverse-map consumption

$$z_t = \sum_r w_{t,r} y_{t,r}, \quad \text{summed only over valid, in-range route slots.}$$

- Require every nonnegative fused position to satisfy the route-map invariant before any arithmetic runs.

Closing the loop: reduction as inverse-map consumption

$$z_t = \sum_r w_{t,r} y_{t,r}, \quad \text{summed only over valid, in-range route slots.}$$

- Require every nonnegative fused position to satisfy the route-map invariant before any arithmetic runs.
- Skip invalid slots *before* accumulating, never after — a zeroed contribution is not the same as an absent one.

Closing the loop: reduction as inverse-map consumption

$$z_t = \sum_r w_{t,r} y_{t,r}, \quad \text{summed only over valid, in-range route slots.}$$

- Require every nonnegative fused position to satisfy the route-map invariant before any arithmetic runs.
- Skip invalid slots *before* accumulating, never after — a zeroed contribution is not the same as an absent one.
- Apply row scales, then any output scale, then cast once.

Closing the loop: reduction as inverse-map consumption

$$z_t = \sum_r w_{t,r} y_{t,r}, \quad \text{summed only over valid, in-range route slots.}$$

- Require every nonnegative fused position to satisfy the route-map invariant before any arithmetic runs.
- Skip invalid slots *before* accumulating, never after — a zeroed contribution is not the same as an absent one.
- Apply row scales, then any output scale, then cast once.

The reduction is trustworthy only when the inverse maps and route weights it consumes travel unmodified from the mapping stage — it does not get to invent its own bookkeeping.

Same ownership rule, four different operators

Surface	What its contract must name
Engram	table address, saved gate state, one gradient-reduction owner
mHC	mix orientation, composed state transition, BF16 reload point
Transpose	exact index bijection, restricted to declared shapes and strides

A modeling wrapper fixes reshaping, dtype/contiguity checks, saved tensors, optional output buffers, and autograd return order — the Engram wrapper chooses returned weight gradients vs. in-place `main_grad` accumulation; the mHC wrapper coordinates several residual-gradient branches through shared storage.

Sharing TileLang or a PyTorch wrapper does not make two contracts interchangeable, and each wrapper gradient contribution has exactly one owner — returned through autograd or accumulated into a shared buffer, never both.

From concept to code: the gate equation, as vectorized PyTorch

The compact gate relation from Section 7 is not a paraphrase — it is the vectorized reference the repository's own tests check kernels against.

```
from TileKernels/tile_kernels/torch/engram.py

raw_dot = torch.einsum('...d,...d->...', x * wh, k_f * we)
dot = raw_dot * rstd_x * rstd_k * scalar
signed_sqrt = dot.abs().clamp_min(clamp_value).sqrt() * dot.sign()
gate_score = signed_sqrt.sigmoid()
output = x + gate_score.unsqueeze(-1) * v.unsqueeze(-2)
```

From concept to code: the gate equation, as vectorized PyTorch

The compact gate relation from Section 7 is not a paraphrase — it is the vectorized reference the repository's own tests check kernels against.

```
from TileKernels/tile_kernels/torch/engram.py

raw_dot = torch.einsum('...d,...d->...', x * wh, k_f * we)
dot = raw_dot * rstd_x * rstd_k * scalar
signed_sqrt = dot.abs().clamp_min(clamp_value).sqrt() * dot.sign()
gate_score = signed_sqrt.sigmoid()
output = x + gate_score.unsqueeze(-1) * v.unsqueeze(-2)
```

`raw_dot` is q ; `dot` is z ; `gate_score` is g ; the final line is literally $y = x + gv$ — the equation, in tensor form, not an approximation of it. [INSPECTED: TileKernels tile_kernels/torch/engram.py]

Testing, evidence, and the chapter recap

What the H200 campaign does and does not establish

Surface	Sampled result	Boundary
Engram hash / top-2-sum gate	2/2, 3/3 rows	one pinned revision, one H200

What the H200 campaign does and does not establish

Surface	Sampled result	Boundary
Engram hash / top-2-sum gate	2/2, 3/3 rows	one pinned revision, one H200
E5M6 cast / cast-back	16/16, 16/16 rows	correctness only, no throughput claim

What the H200 campaign does and does not establish

Surface	Sampled result	Boundary
Engram hash / top-2-sum gate	2/2, 3/3 rows	one pinned revision, one H200
E5M6 cast / cast-back	16/16, 16/16 rows	correctness only, no throughput claim
Fused SwiGLU backward + cast	48/72 rows	24 fixture-setup failures, not target-kernel failures

What the H200 campaign does and does not establish

Surface	Sampled result	Boundary
Engram hash / top-2-sum gate	2/2, 3/3 rows	one pinned revision, one H200
E5M6 cast / cast-back	16/16, 16/16 rows	correctness only, no throughput claim
Fused SwiGLU backward + cast	48/72 rows	24 fixture-setup failures, not target-kernel failures

A direct TileKernels test exercises the kernel or wrapper it names. A serving runtime uses that surface only when its call path selects the same function and satisfies its layout, dtype, and fallback conditions — tests do not imply runtime adoption.

Concept-to-code map for the chapter

- **quantization casts and scale metadata** → `TileKernels/tile_kernels/quant/`
- **MoE gating, mapping, and expansion** → `TileKernels/tile_kernels/moe/`
- **route-map reduction, as a local oracle** →
`code/chapter_tilekernels_tilelang_kernel_surfaces/moe_route_map_reference.py`
- **Engram address, gate, and gradient ownership** → `TileKernels/tile_kernels/engram/`
- **mHC component and composed transitions** → `TileKernels/tile_kernels/mhc/`
- **transpose launch-domain enforcement** → `TileKernels/tile_kernels/transpose`

Preserve identity through every reorder, pack, and reduction — or a plausible tensor is just a wrong answer with good posture.

One sentence, and what carries forward

Ownership survives the transform, or the numbers do not mean what they appear to mean.

- Six named invariants — across quantization, gating, mapping, reduction, and the three adjacent surfaces — are this chapter's real product, alongside its diagrams and worked numbers.
- No blanket production claim follows: reported, inspected, derived, and observed evidence each cover a distinct, bounded part of the repository.

Next: the parallelism-taxonomy chapter carries these token, expert, and rank-ownership invariants into multi-rank execution, where operator-local correctness becomes one part of a larger placement and communication invariant.